

Chapter 3 — Frontend System Design and Implementation

3.1 Introduction

The frontend subsystem of the Briefly Customer Relationship Management (CRM) platform represents the primary interaction interface between system users—including e-commerce business operators, managers, support agents, and organization administrators—and the underlying backend service architecture. Engineered as a modern Single Page Application (SPA), the Briefly frontend handles complex data visualization, real-time analytics telemetry, customer support ticket management, and multi-tenant organization control workflows.

In a multi-tenant customer relationship management (CRM) ecosystem, the client interface must reconcile two opposing engineering goals: displaying high-throughput analytical query updates while maintaining an intuitive, responsive, and visually cohesive user experience. The Briefly client addresses these challenges by decoupling the state coordination layer from the rendering pipeline, enforcing strict domain isolation, and utilizing a custom-designed, token-driven design system.

The primary responsibilities of the frontend subsystem are structured as follows:

1. **Data Presentation and Control:** Rendering multi-faceted tables, detailed customer profile records, and integration settings templates using custom component abstractions.
2. **State Orchestration and Caching:** Managing client-side session states, application interface layouts, and cached server responses to minimize network overhead and database load.
3. **Session Security and Routing:** Enforcing Role-Based Access Control (RBAC) across public, guest, and protected routing hierarchies.
4. **Integration Telemetry:** Providing clean workflows to connect, synchronize, and monitor external platforms, such as Shopify.

By prioritizing academic software engineering principles—specifically the Separation of Concerns (SoC) and Feature Isolation—the Briefly frontend ensures that the application remains maintainable, scalable, and secure as the platform expands.

3.2 Frontend Requirements Analysis

3.2.1 Functional Requirements

The functional specifications of the Briefly frontend are derived directly from the active domain modules in the codebase. These modules correspond to the business logic containers necessary for e-commerce CRM operations:

- **Authentication and Authorization:** Users must be able to securely sign up, log in, perform multi-tenant onboarding, and access dashboard resources matching their assigned roles (e.g., Administrator, Manager, Agent).

- **Multi-Tenant Organization Management:** Administrators must have the capability to update their organization's identity parameters (name, slug, and brand logo asset) or execute tenant deletion procedures subject to strict security confirmations.
- **Customer Intelligence:** The system must display customer profiles containing detailed metadata (email, phone, geographic location, and marketing consent) alongside advanced metrics such as Recency, Frequency, Monetary (RFM) segments, cohort groups, lifetime value (LTV) scores, and churn risk indices. Additionally, support for adding customer notes and viewing chronologically ordered event logs is required.
- **Product Management:** Users must have the ability to view, search, and manage products, inspect specific details, and associate product entries with customer transaction records.
- **Order Tracking:** The system must process e-commerce transaction details, including order identifiers, purchase dates, payment statuses (e.g., Paid, Refunded, Pending), shipping statuses (e.g., Processing, Shipped, Delivered), and lifetime spending totals.
- **Support Ticket Systems:** The client interface must facilitate customer service ticket tracking, rendering ticket details, conversation threads, and assignment statuses.
- **Shopify Integration Control:** The frontend must provide a connection panel allowing users to link external Shopify stores by specifying domain names and API credentials, manage data conflict rules (such as "Shopify wins"), and audit real-time synchronization logs.
- **Role-Based Access Control (RBAC) UI:** Administrators must be able to create, edit, or delete custom roles and assign explicit Read, Write, and Delete permissions across primary system resources (Customers, Segments, Campaigns, Products, Tickets, Employees).

3.2.2 Non-Functional Requirements

To support the business operations of multiple concurrent merchants, the frontend is constrained by the following non-functional engineering standards:

- **Performance:** High-density data tables must load in under 1.5 seconds under standard broadband conditions. Data-fetching libraries must implement aggressive query caching, maintaining stale-while-revalidate configurations to avoid redundant network roundtrips.
- **Scalability:** The architecture must adhere to a strict modular folder structure, allowing developers to add features or modify existing business logic without causing regression side-effects in unrelated components.
- **Security:** Sensitive tokens must be stored securely and transmitted via interceptors. Application routes must be guarded client-side to prevent unauthorized rendering of administrative panels.
- **Maintainability:** Coding standards must restrict files to under 200 lines where possible. Complex formatting, validation, and filtering logic must be extracted to isolated utility modules.
- **Responsiveness:** The layout must adapt seamlessly across desktop, tablet, and mobile displays, utilizing collapsible navigation rails and responsive layout grids.
- **Accessibility:** All form elements must be properly labeled, interactive components must

have distinct highlight states, and semantic HTML5 structures must be used to ensure screen-reader compatibility.

3.3 Frontend Technology Stack

The technological foundation of the Briefly frontend consists of modern, library-supported tools selected to maximize runtime performance and minimize build times.

Technology	Purpose	Justification
React 19.2.4	Core UI Component Framework	Provides declarative component rendering, concurrent rendering pipelines, and efficient DOM reconciliation.
TypeScript 6.0.2	Static Type Safety and Contracts	Enforces compile-time type verification, minimizing runtime exceptions and establishing clear interface contracts.
Vite 8.0.4	Build Tool and Dev Server	Replaces slower bundlers with native ESM-based hot module replacement (HMR), reducing development build cycles.
Zustand 5.0.12	Client-Side Global State Management	Offers a lightweight, flux-like store architecture with automatic local storage persistence, avoiding the boilerplate of Redux.
TanStack Query v5.100.6	Server State and Cache Management	Eliminates manual <code>useState</code> and <code>useEffect</code> data-fetching loops, providing automatic query invalidation and loading state tracking.
Axios 1.15.2	HTTP Client	Simplifies API communication with built-in support for request/response interceptors, cookie management, and standard JSON transformation.
Tailwind CSS 4.2.2	Utility-First Styling Framework	Accelerates UI implementation with compile-time utility classes, ensuring design consistency and eliminating unused CSS rules.
Better Auth 1.6.9	Authentication Integration	Provides standard client helpers to connect seamlessly with modern oauth/session-based backend validation.

Technology	Purpose	Justification
Formik 2.4.9	Form State Management	Handles complex form state validation, field tracking, and submission states with low performance overhead.
Yup 1.7.1	Schema Validation	Enables declarative validation schemas that link with Formik to validate client inputs before API transmission.
Framer Motion 12.40.0	Interactive Micro-animations	Handles hardware-accelerated transitions, sliding overlays, and modal scaling to create a polished user experience.
Hugeicons React 0.4.0	Consistent Graphic Iconography	Supplies clean, modern SVG vector assets that fit the system's professional aesthetic.

3.4 Frontend Architecture and Project Directory Structure

The architectural design of the Briefly client conforms to the **Feature-Isolated Layered Pattern**. Rather than grouping files by technical role (e.g., all hooks in one folder, all components in another), Briefly isolates code by business domain (e.g., Customers, Orders, Campaigns) under `src/features/`. Each domain is a self-contained module containing its own services, hooks, type definitions, utility helpers, and subcomponents. Shared structural components reside in `src/core/`.

This approach ensures a unidirectional dependency flow: page controllers import from their local feature modules, and feature modules import shared code from the `core/` layer. Direct cross-imports between independent feature directories are strictly prohibited. If two features must share logic, that logic is extracted to the `core/` directory, preventing cyclic dependency graphs.

The physical layout of the codebase maps directly to this conceptual framework:

```

src/
├── api/
│   ├── client.ts           # Global network infrastructure and client instance
│   └── endpoints/
│       └── endpoints.ts    # Axios client instance with token configuration
├── app/                    # Centralized API endpoints map
├── router                  # Top-level shell bootstrap, providers, and main
│   ├── App.tsx             # Main application component
│   ├── providers.tsx       # Global context providers (React Query, UI, Toast)
│   └── router.tsx          # Consolidated routing map
├── assets/                 # Structural SVGs, logo assets, and custom icon
├── definitions
├── core/                   # Reusable, feature-agnostic infrastructure
│   ├── components/        # Generic UI widgets (DataTable, PageLayout, Modal,
│   │                       etc.)
│   ├── layouts/           # Dashboard shell layout structure (Sidebar + Navbar)
│   └── types/              # Global interface contracts and API definitions

```

└─ hooks/	# Shared hooks (e.g., useAuth for session extraction)
└─ features/	# Self-contained business domain modules
└─ customers/	# Customer profiles, notes, list page, hooks,
services	
└─ orders/	# Transaction listing, status updates, detail view
└─ segments/	# Customer cohort filtering and rule definition
└─ settings/	# Multi-tenant settings, RBAC profiles, integrations
└─ store/	# Zustand global client-side state engines
└─ auth.store.ts	# Authentication session state
└─ ui.store.ts	# Sidebar navigation state
└─ pages/	# Public entry views (landing, signup, login)

The data lifecycle within this architecture moves from the user interface down through queries, services, and network layers.

```
graph TD
    UI[Presentation Layer: Pages & Components]
    Core[Core Components: PageLayout & DataTable]
    Store[Global Client State: Zustand Stores]
    Hooks[Feature Hooks: TanStack Query]
    Services[Feature Services: Axios API Clients]
    API[Network Layer: Axios Instance & Interceptors]

    UI --> Core
    UI --> Store
    Core --> Hooks
    Hooks --> Services
    Services --> API
```

Figure 3.1: Unidirectional Data Flow and Layered Dependency Hierarchy

3.5 User Interface and User Experience Design Strategy

The Briefly design philosophy emphasizes clarity, information density, and interactive responsiveness. The visual design avoids standard default styles, using a custom color system and modern typography to establish a clean, professional aesthetic.

3.5.1 Typography and Color System

The visual presentation uses two primary font families from Google Fonts, imported via `src/index.css`:

- **Poppins:** Used as the primary typeface for body text, form elements, table contents, and navigation menus to ensure legibility across all screen sizes.
- **Parkinsans:** Used selectively for primary numeric statistics, metrics headings, and analytics telemetry readouts.

The platform's color system is defined using CSS variables in the `:root` scope, allowing for consistent branding and potential theme extensions:

- **Primary Brand Colors:** `--color-primary-500` (`#4B91E2`) serves as the system's signature blue for primary buttons, selection indicators, and active sidebar states. The shades range from `--color-primary-100` (`#D3E4F8`) for background highlights to `--color-primary-900` (`#07182C`) for high-contrast text elements.

- **Status Indicators:** Clear status communication is achieved using specific semantic color codes: `--color-success` (`#22C55E`), `--color-warning` (`#F59E0B`), and `--color-error` (`#EF4444`).
- **Background Surfaces:** The main workspace uses a soft slate background (`#F8FAFC`), while cards, data grids, and modals use a solid white surface (`#FFFFFF`) framed by a subtle gray border (`#E5E7EB`).

3.5.2 Layout Patterns and Workflows

- **Collapsible Sidebar Navigation:** The application sidebar uses a custom width allocation model. On large displays, it adjusts between an expanded layout of `14.72vw` (or `212px`) and a collapsed rail of `70px` that displays only icons. On smaller screens, the sidebar transitions to a sliding overlay drawer controlled by Zustand state flags.
 - **Metric Grids:** Dashboards and detail views present key performance indicators in a responsive grid. For example, the customer profile view organizes LTV, average order values, ticket volume, and shopping cart abandonment rates into a two-column grid.
 - **Organization Settings Panel:** To manage multi-tenant details, the system provides a tabbed settings interface. This allows administrators to adjust brand identifiers, configure synchronization mappings, and set up role assignments within a single view.
 - **Role-Based Access Control Interface:** The RBAC interface presents a grid mapping resource types to operations (Read, Write, Delete). This visual matrix lets administrators configure granular permissions for custom roles, with updates saved directly to the database via API calls.
-

3.6 Declarative Routing and Route Protection System

Routing within the Briefly client is managed by the React Router library (specifically `react-router-dom` v7). The routing architecture separates public entry views from the secure dashboard environment.

Security is enforced using two route wrapper components:

1. **ProtectedRoute:** Restricts access to authenticated users. If no session token is found, it redirects the browser to `/login` while preserving the requested path in the router state. If the user's onboarding flow is incomplete, it redirects them to `/onboarding`.
2. **GuestRoute:** Restricts access to unauthenticated users. Logged-in users attempting to access public login or registration pages are automatically redirected back to `/dashboard`.

The route mapping hierarchy is defined in `src/app/router.tsx`:

```
export default function AppRouter() {
  return (
    <Router>
      <Routes>
        { /* Public Routes */ }
        <Route path="/" element={ <LandingPage /> } />
      </Routes>
    </Router>
  )
}
```

```

        { /* Guest-Only Routes */ }
        <Route path="/login" element={<GuestRoute><Login
/></GuestRoute>} />
        <Route path="/signup" element={<GuestRoute><Signup
/></GuestRoute>} />

        { /* Protected Routes */ }
        <Route path="/onboarding"
element={<ProtectedRoute><Onboarding /></ProtectedRoute>} />
        <Route path="/dashboard/*"
element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />
    </Routes>
  </Router>
);
}

```

Within the dashboard, nested routes are resolved dynamically inside the main layout controller (src/core/layouts/Dashboard.tsx). Dynamic URL parameters (such as customers/:id) are extracted by detail views to fetch specific records:

```

const Dashboard = () => {
  return (
    <div className="flex h-screen overflow-hidden bg-white">
      <Sidebar />
      <div className="flex-1 flex flex-col min-w-0 overflow-hidden bg-[#F8FAFC]">
        <Navbar />
        <main className="flex-1 overflow-y-auto p-4 md:p-6 lg:p-8">
          <Routes>
            <Route path="/" element={<DashboardHome />} />
            <Route path="customers" element={<Customers />} />
            <Route path="customers/:id" element={<CustomerProfile />} />
            <Route path="segments" element={<Segments />} />
            <Route path="campaigns" element={<Campaigns />} />
            <Route path="products" element={<Products />} />
            <Route path="orders" element={<Orders />} />
            <Route path="tickets" element={<Tickets />} />
            <Route path="employees" element={<Employees />} />
            <Route path="settings" element={<Settings />} />
          </Routes>
        </main>
      </div>
    </div>
  );
};

```

To restrict layout access dynamically before the component hierarchy mounts, Listing 3.1 displays the core implementation of the protected route interceptor.

```

// Listing 3.1: ProtectedRoute.tsx route guard logic
import { Navigate, useLocation } from "react-router-dom";
import { useAuthStore } from "@store/auth.store";

export function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const token = useAuthStore((s) => s.token);
  const onboardingComplete = useAuthStore((s) => s.onboardingComplete);
  const location = useLocation();

  if (!token) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }
}

```

```

    if (!onboardingComplete && location.pathname !== "/onboarding") {
      return <Navigate to="/onboarding" replace />;
    }

    return <{children}</>;
  }

```

3.7 Client-Side State Management Strategy

To ensure high performance and clean separation of concerns, Briefly categorizes application state into three distinct layers:

3.7.1 Local UI State

Component-specific interactive states (such as active dropdown flags, input fields, and modal visibility states) are managed using React's native `useState` hook. This keeps temporary interface states isolated within their respective components and avoids polluting global stores.

3.7.2 Global Client State

Global client-side states that persist across page transitions are managed using Zustand. The application defines two primary Zustand stores:

- **`useAuthStore (src/store/auth.store.ts)`:** Stores session tokens, user profiles, current organization roles, and permissions. This store is persisted to `localStorage` using Zustand's `persist` middleware, ensuring user sessions survive browser refreshes.
- **`useUIStore (src/store/ui.store.ts)`:** Manages interface layout states, such as whether the navigation sidebar is collapsed or expanded.

Listing 3.2 illustrates the structure of the authentication state manager:

```

// Listing 3.2: auth.store.ts Zustand store configuration
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";
import type { AuthUser, AuthState } from "../types";

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      token: null,
      user: null,
      role: null,
      permissions: null,
      onboardingComplete: false,

      setSession: (token, user, role, permissions, onboardingComplete =
false) =>
        set({ token, user, role, permissions, onboardingComplete }),

      completeOnboarding: () =>
        set({ onboardingComplete: true }),

      clearSession: () =>
        set({ token: null, user: null, role: null, permissions: null,

```

```
onboardingComplete: false })),
    },
    {
      name: "briefly-auth",
      storage: createJSONStorage(() => localStorage),
    }
  )
);
```

3.7.3 Server State

Server-side data synchronization is managed entirely by TanStack Query. Rather than using manual fetching patterns, components retrieve data through custom React hooks that call the underlying service API.

To optimize caching and reduce server load, TanStack Query is configured globally in `src/app/providers.tsx`:

- **staleTime:** Set to 5 minutes ($1000 * 60 * 5$). During this window, cached data is considered fresh, preventing duplicate API requests.
- **refetchOnWindowFocus:** Disabled (`false`) to prevent unnecessary background queries when switching browser tabs.
- **Targeted Mutation Invalidation:** When modifications are made (such as creating or deleting a customer), the corresponding query keys are invalidated. This triggers an automatic, background update of the UI.

3.8 Authentication and Authorization Subsystem

Briefly secures sessions and manages client-side access control through a combined architecture of session tokens and granular permissions:

3.8.1 Session Lifecycle and Token Management

1. **Login and Registration:** Users authenticate via `/login` or `/signup`. The API returns a secure token, user metadata, active role definitions, and permission tables.
2. **Session Persistence:** The credentials are saved to `useAuthStore` using the `setSession` action, initializing the user session and redirecting them to the dashboard.
3. **Authentication Injection:** An Axios request interceptor injects the active token into the `Authorization` header of every outgoing API call as a Bearer token.
4. **Token Expiry and Logout:** If an API call returns a `401 Unauthorized` response, a response interceptor catches the error, clears the active session state, and redirects the user to the login page.

3.8.2 Role-Based Access Control (RBAC)

Briefly implements a granular authorization model. Users are assigned roles (e.g., `role-admin`, `role-manager`, `role-agent`), and each role contains a permission map that matches resources

to actions (read, write, delete).

The platform maps permissions to a resource-based format: `Resource.action` (e.g., `Customer.create`, `Campaign.send`, `Setting.write`). This configuration is retrieved from the `/auth/get-session` endpoint upon login and cached within the persistent Zustand store. The frontend uses this model to hide or show navigation items, action buttons, and entire columns.

To enforce authorization rules in the UI, the frontend uses a custom `<PermissionGuard>` component, shown in Listing 3.3.

```
// Listing 3.3: PermissionGuard.tsx access check logic
import { useAuthStore } from "@store/auth.store";

type PermissionGuardProps = {
  permission: string;
  fallback?: React.ReactNode;
  children: React.ReactNode;
};

export function PermissionGuard({ permission, fallback = null, children }:
PermissionGuardProps) {
  const permissions = useAuthStore((s) => s.permissions) || {};
  const [resource, action] = permission.split(".");

  if (!resource || !action) {
    console.warn(`[PermissionGuard] Invalid permission format: "${permission}"`);
    return <{fallback}</>;
  }

  const resourcePermissions = permissions[resource];
  const hasPermission = resourcePermissions?.includes(action);

  if (!hasPermission) {
    return <{fallback}</>;
  }

  return <{children}</>;
}
```

```
sequenceDiagram
    autonumber
    actor User as Client User
    participant Router as AppRouter
    participant Guard as ProtectedRoute
    participant Comp as PermissionGuard
    participant Store as Zustand AuthStore
    participant API as Backend API

    User->>Router: Access /dashboard/customers
    Router->>Guard: Evaluate protection
    Guard->>Store: Read token & onboardingComplete
    alt No Token or Onboarding Pending
        Store-->>Guard: Unauthenticated
        Guard-->>User: Redirect to /login or /onboarding
    else Session Validated
        Store-->>Guard: Access Approved
        Router->>Comp: Mount Page Layout
        Comp->>Store: Verify permission: "Customer.read"
        Store-->>Comp: Has "read" action in "Customer" resource array
```

```

    alt Permission Granted
      Comp-->>User: Render Customer DataTable
    else Permission Denied
      Comp-->>User: Render Fallback Element (Restricted Access)
    end
  end
end

```

Figure 3.2: Authentication and Fine-Grained Authorization Flow

3.9 Centralized API Integration Layer

API communication is managed by a centralized Axios instance (`src/api/client.ts`) configured with a base URL, request timeout rules, and cross-origin credential parameters:

```

// Listing 3.4: client.ts centralized HTTP client
import axios from "axios";
import { useAuthStore } from "@store/auth.store";

const apiClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    "Content-Type": "application/json",
  },
  withCredentials: true,
});

apiClient.interceptors.request.use((config) => {
  const token = useAuthStore.getState().token;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

apiClient.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore.getState().clearSession();
    }
    return Promise.reject(error);
  }
);

export default apiClient;

```

To keep network logic separated from UI rendering, Briefly registers backend routes in a single endpoints map (`src/api/endpoints/endpoints.ts`). Individual services (such as `customerService` or `settingsService`) call these endpoints using Axios, unwrap the response data, and return clean objects to the UI.

The network architecture follows a strict delegation pattern:

1. **Endpoints (`endpoints.ts`):** Supply absolute paths and parameter builders.
2. **Services (`*.service.ts`):** Execute HTTP requests using `apiClient`, returning parsed data models.

3. **Hooks (* .hooks .ts)**: Wrap service methods in TanStack Query, managing caching rules, loader state, and query invalidation.
4. **UI Components (* .tsx)**: Call hooks and render the returned states, avoiding direct backend integrations.

```
sequenceDiagram
    autonumber
    actor User as Client Browser / User
    participant FE as Frontend UI (React Component)
    participant Store as Zustand Auth Store
    participant Interceptor as Axios Interceptor (apiClient)
    participant BE as Backend API Endpoint
    participant DB as CRM Database

    User->>FE: Request Customer Profile
    FE->>Interceptor: GET /customers/:id (via Customer Service)
    activate Interceptor
    Interceptor->>Store: Read Bearer Token
    Store-->>Interceptor: Return Token
    Note over Interceptor: Inject Token into Authorization Header
    Interceptor->>BE: GET /api/customers/:id
    activate BE
    BE->>DB: Query Customer Data
    DB-->>BE: Customer Records & Analytics
    BE-->>Interceptor: JSON Response (200 OK)
    deactivate BE
    Interceptor-->>FE: Clean Data (Unwrapped)
    deactivate Interceptor
    FE-->>User: Render Customer Profile & Telemetry
```

Figure 3.3: Frontend–Backend Communication and Token Injection Flow

3.10 Data Query Caching and React Mutation Patterns

Data fetching is engineered to follow a declarative lifecycle. Instead of components manually managing loading animations, error messages, and network triggers inside `useEffect` wrappers, they consume custom React hooks.

To demonstrate how the frontend communicates with the service layer, Listing 3.5 outlines the React Query hooks mapped to the customer domain:

```
// Listing 3.5: customer.hooks.ts custom data-fetching hooks
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { customerService } from "../customer.service";
import toast from "react-hot-toast";

export const customerKeys = {
  all: ["customers"] as const,
  list: () => [...customerKeys.all, "list"] as const,
  detail: (id: string) => [...customerKeys.all, "detail", id] as const,
};

export const useCustomers = () =>
  useQuery({
    queryKey: customerKeys.list(),
    queryFn: customerService.getAll,
  });
```

```
export const useCreateCustomer = () => {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (payload: Record<string, unknown>) =>
customerService.create(payload),
    onSuccess: () => {
      qc.invalidateQueries({ queryKey: customerKeys.all });
      toast.success("Customer created!");
    },
    onError: (err: any) => {
      toast.error(err?.response?.data?.message || "Failed to create
customer");
    },
  });
};
```

This model provides several architectural benefits:

- **Query Key Factories:** Grouping query structures in `customerKeys` creates a predictable namespace. Invalidation triggers like `qc.invalidateQueries({ queryKey: customerKeys.all })` automatically clear both list caches and specific item detail caches.
- **Optimistic UI Updates:** Using standardized lifecycle callbacks allows the client to update elements immediately, reverting to previous states only if server mutations fail.
- **Automatic Error Boundary Notifications:** By hooking into the `onError` callbacks, errors are normalized and pushed to user-friendly UI toasts automatically, eliminating manual try-catch wrappers inside components.

3.11 Performance Optimization, Responsiveness, and Accessibility

The Briefly client uses several optimization techniques to ensure the UI remains fast and responsive, even when handling large datasets.

3.11.1 Client-Side Performance Optimizations

- **Stale-While-Revalidate Caching:** Powered by React Query, the application serves stale data from the local cache immediately, while requesting fresh updates in the background. This eliminates loading screens during navigation.
- **Native Image Lazy Loading:** The shared image component (`src/core/components/Image.tsx`) uses native browser lazy loading (`loading="lazy"`). This delays loading images that are off-screen until the user scrolls near them, reducing initial page load times and saving bandwidth.
- **Dynamic Rendering with Virtual Tables:** The generic `DataTable` component uses pagination client-side or server-side to limit DOM size. By rendering only active rows, the application maintains a low memory footprint and ensures smooth scrolling and layout calculations.

- **Code Splitting and Type Safety:** The build pipeline uses TypeScript's `verbatimModuleSyntax` standard. This ensures that type imports are stripped out during compilation, reducing bundle sizes and preventing type definitions from leaking into the output code.

3.11.2 Responsive Layout Strategy

The layout is designed using mobile-first grid configurations:

- **Fluid Layout Grids:** The interface uses Tailwind CSS's flexbox and grid layouts. For example, columns in the settings and profile sections wrap dynamically:

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
```

On mobile devices, this layout stacks fields vertically to prevent horizontal scrolling, while automatically expanding to multi-column grids on wider screens.

- **Responsive Sidebar Transitions:** The navigation sidebar utilizes media queries to adjust its display. On screens below 1024px, it transitions from a persistent sidebar to a sliding drawer controlled by global layout state flags. This drawer slides in over a dark overlay (`bg-black/40`) to maintain focus, and closes automatically when a navigation item is selected.

3.11.3 Accessibility Features

- **Keyboard Navigation:** Custom interactive elements (such as modally presented forms) support standard keyboard controls. Checked lists use native HTML input types (`type="checkbox"`) to preserve keyboard focus indicators.
- **Semantic HTML Structure:** Pages are organized using structural HTML5 tags (like `<nav>`, `<aside>`, `<main>`, and `<header>`), allowing screen readers to parse the layout structure.
- **Visual Contrast:** High-contrast text values (such as charcoal `text-gray-900` on off-white backgrounds) ensure the interface meets readability standards for visually impaired users.

3.12 System Challenges and Technical Solutions

Developing Briefly's frontend highlighted several challenges in managing complex multi-tenant systems.

3.12.1 React 19 Integration with Form Library APIs

Integrating React 19 with Formik required resolving compatibility conflicts with React's new virtual DOM rendering mechanisms. Formik occasionally failed to capture state updates inside dynamic dropdown fields when running alongside React 19's asynchronous updates.

This issue was addressed by implementing custom form change hooks. Instead of relying on Formik's default change handlers for nested structures, inputs execute manual values updates using

`formik.setFieldValue()`. This ensures form fields sync correctly before execution logic starts.

3.12.2 Managing Cache States in Large-Scale Environments

In multi-tenant setups, one tenant might modify database entries while another views stale dashboards. This mismatch was resolved by using strict cache staleness boundaries in TanStack Query. Critical lists, such as transaction records and customer notes, are configured with a `staleTime` of zero. This forces components to fetch fresh background updates on focus, preventing data overlap across different tenants.

3.13 Future Work

The platform's roadmap includes two major technical enhancements to expand its capabilities:

3.13.1 Real-Time Telemetry via WebSockets/SSE

To display events and integration updates instantly without manual page refreshes, the application will integrate a real-time communication layer. Replacing short-polling loops with WebSockets (using `socket.io-client` or native WebSockets) will allow the backend to push events immediately to a unified state listener. This will let managers monitor customer interactions and incoming support tickets in real-time.

```
[WebSocket Client Hub] <--- Event Payload (JSON) --- [WebSocket Server]
      |
      |> Dispatch UPDATE_TICKET —> Refresh active tickets view
      |> Trigger toast notification —> Alert active agent
```

3.13.2 Progressive Web Application (PWA) Support

To improve access on mobile devices, the Briefly client will be updated to support PWA capabilities. This includes adding service worker scripts to cache key assets (styles, navigation routes, and core bundles) and setting up a manifest configuration for offline access. Offline storage will capture edits locally in IndexedDB and synchronize them with the server once connection is restored, ensuring a reliable user experience even in poor network conditions.

3.14 Conclusion

Chapter 3 presented the frontend system design and implementation of the Briefly CRM platform. By choosing React 19, Vite, Zustand, and TanStack Query, the application establishes a reliable framework for handling complex e-commerce analytics.

Separating domain-specific components into isolated modules prevents dependency cycles, while custom HTTP client interceptors and role guards secure system access. This modular frontend architecture ensures the platform remains performant and easy to maintain as Briefly scales.